

# Placement Portal Application Report

## 1. Student Details

**Name:** Chandan kumar

**Roll Number:** 23f1003153

**Email:** 23f1003153@ds.study.iitm.ac.in

**About Me:** I am a student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience. This project demonstrates the design and development of a placement portal using Flask and SQLAlchemy, focusing on role-based access and application management.

## 2. Project Details

### Problem Statement:

Students often face difficulty tracking placement opportunities and managing applications across multiple companies. Similarly, companies need an organized way to evaluate candidates and manage recruitment drives. A centralized platform is required to track these activities and provide clear insights into application status.

### Approach:

The application is built using **Flask and SQLAlchemy** to create a web-based system where students can track available drives and their application status, while companies can manage recruitment drives and review applicants. The system uses role-based access for students, companies, and administrators to ensure proper management and workflow of the placement process.

## 3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

## 4. Technologies Used

### Backend

- Python : web framework for python used to build the backend of the application.
- Flask : web framework for python used to build the backend of the application.
- Flask-SQLAlchemy : Integrates SQLAlchemy into Flask for DB operations.
- SQLAlchemy : ORM(Object Relation Mapping) to interact with the relational databases.

### Frontend

- HTML : Structure and markup of web pages.
- Bootstrap (for UI styling) : CSS framework for responsive design.
- Jinja2 Templates : Template engine for python.

### Database

- SQLite (for storing user, company, student, drive, and application data): Embedded SQL database used via SQLAlchemy.

## 5. DB scheme design

1. User
  - Stores authentication and role information
  - Columns are id,username,password,email,role,name,is\_approved,is\_blacklisted
  - Central user management table
  - Different roles determine system access.
2. Company\_Profiles
  - Columns are id, user\_id, company\_name, description
  - Relationship : one user → one company profile
  - Stores company specific details.
3. Student\_Profiles
  - Columns are id, user\_id, department, resume, skills
  - Relationship : One user → one student profile
  - Stores student academic and skill information
4. Drive
  - Columns are id, company\_id, job\_title,job\_description, salary, location, eligibility, deadline, status, approval\_status,created\_at
  - Stores recruitment drive details created by companies.
5. Application
  - Columns are id, student\_id, drive\_id, date\_applied, status, remark
  - Unique constraint on (student\_id, drive\_id)ensures a student cannot apply twice for the same drive.
  - Tracks student applications and their recruitment status.



## 6. API Design

### 1. Authentication

- User Login : /api/login
- Student registration : /api/student\_signup
- Company registration : /api/company\_signup
- Logout : /api/logout

### 2. Admin API

- For admin dashboard : /api/admin\_dashboard
- For searching student and company : /api/search
- For giving approval to company drive : /api/approve\_drive/<int:drive\_id>
- For rejecting Company Drive : /api/reject\_drive/<int:drive\_id>
- For view student application : /api/student\_application/<int:application\_id>
- For view drive details : /api/drive\_details/<int:drive\_id>
- For approve company registration: /api/approve\_company/<int:user\_id>
- For blacklist user : /api/blacklist\_user/<int:user\_id>

### 3. Student API

- For Student Dashboard : /api/student\_dashboard
- For editing their profiles : /api/edit\_student\_profile
- For view their application : /api/my\_applications
- For view company drive : /api/company\_details/<int:drive\_id>
- For view company details : /api/company\_details/<int:user\_id>
- For view drive details : /api/drive\_details/<int:drive\_id>
- For apply for company drive : /api/apply/<int:drive\_id>

### 4. Company API

- For company dashboard : /api/company\_dashboard
- For mark as complete for drive : /api/complete\_drive/<int:drive\_id>
- For update close drive : /api/update\_closed\_drive/<int:drive\_id>
- For update application drive : /api/update\_application/<int:drive\_id>
- For view student application : /api/student\_application/<int:application\_id>
- For update student application status :  
/api/update\_application\_status/<int:application\_id>/<string:status>
- For create drive : /api/create\_drive
- For remove drive : /api/remove\_drive/<int:drive\_id>

## 7. Architecture and Features

### 1. backend/ :

- [models.py](#) : contains database models using sqlalchemy
- [database.py](#) : contain import of sqlalchemy
- [controllers.py](#) : contain business logic, manages frontend-related route handling, rendering HTML pages.

### 2. instance/ :

- Holds sqlite database file : `placementportal.sqlite3`

### 3. static/ :

- Contains css files that styles various HTML pages

4. templates/ :
  - All the HTML pages are placed here.
5. venv :
  - Contain virtual environment files.
6. Gitignore :
  - Contains files to exclude from git.
7. [app.py](#) :
  - This is the main entry point of the app. It initializes the Flask app, configures the database, sets up login functionality and registers the routes and APIs.

## Core Features :

1. Authentication :
  - There are two types signup : Student and Company
  - Login are same for student, admin, company
  - Admin are pre-exist, admin can only login
2. Student Dashboard:
  - View company details, view their ongoing drive and apply for it.
  - View their application status
  - View their History and can edit their profiles
3. Admin Dashboard :
  - Approve company, approve or reject their drives and blacklist company and student
  - Total number of students, total number of company, total number of drive, total number of applications.
  - Here also search by username of company and students
  - Admin can view company ongoing drive and students applications
4. Company Dashboard:
  - Company can create, update and remove drive.
  - Here all showed all ongoing drive, upcoming drive, closed drive, rejected drive
  - Company can view number of applicants are applied for drive or they can select, reject or shortlist applications.

## 8. Video Presentation

Drive Link:  Screen Recording 2026-03-09 144822.mp4